SLIIT
Discover Your Future

IT1170 – Data Structures and Algorithms

Lecture 09

Divide and Conquer - 1

SLIIT
Discover Your Future

Today's Lecture

- Divide and Conquer
- Applications
  - Quick Sort
  - Merge Sort
- Analysis

Divide and Conquer

Divide: Break the problem into sub problem recursively.

Conquer: Solve each sub problems.

Combine: All the solutions of sub problems are combined to get the solution of the original problem.

Applications

- Quick Sort
  - More work on divide phase.
  - Less work for others.
- Merge Sort
  - Vice versa of Quick sort.

Quick Sort (contd.)

Divide: Partition (rearrange) the array A[p..r] into two (possibly empty) sub arrays A[p..q - 1] and A[q + 1..r]

- Each element of A[p..q - 1] is less than or equal to A[q]
- Each element of A[q + 1..r]is greater than or equal to A[q].
- Compute the index q as part of this partitioning procedure.

Conquer: Sort the two subarrays A[p..q -1] and A[q +1..r] by recursive calls to quicksort.

Combine: Since the sub arrays are sorted in place, no work is needed to combine them: the entire array A[p..r] is now sorted.

Quick Sort procedure

Input: Unsorted Array (A,p,r)

Output: Sorted sub array A(1..r)

```
QUICKSORT (A,p,r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT (A, p, q-1)
    QUICKSORT (A, q+1, r)
```

To sort an entire array A, the initial call is

QUICKSORT(A, 1, A.length).

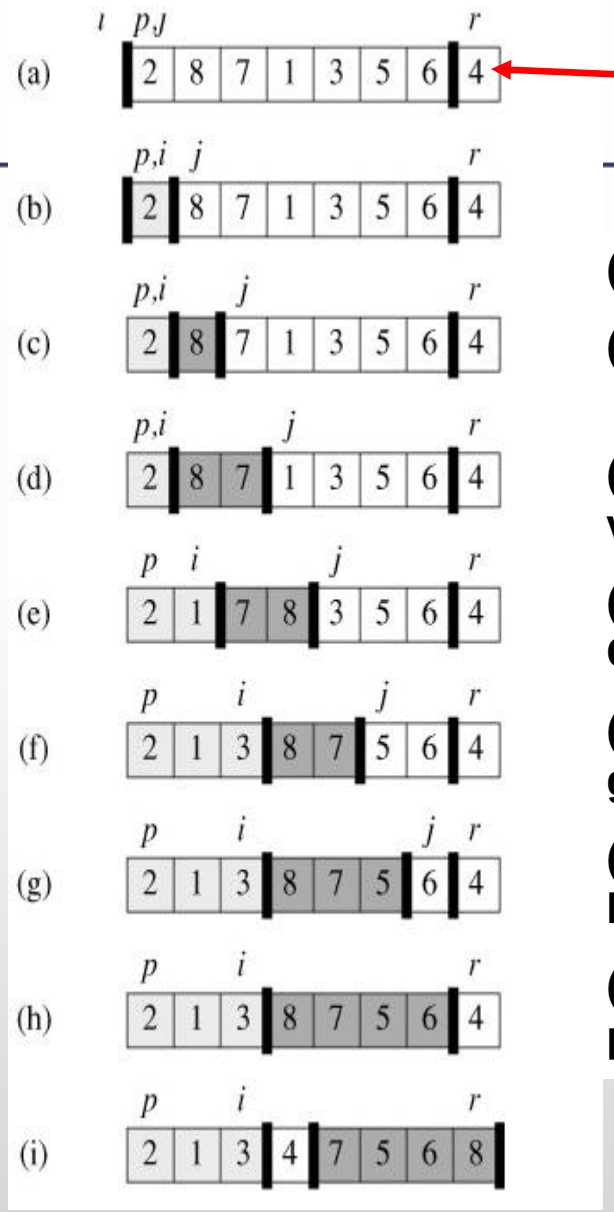
Partition Algorithm

PARTITION(*A*, *p*, *r*)

```
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

The key to the algorithm is the PARTITION procedure, which rearranges the sub array A [p..r] in place.

Operation of PARTITION on an 8-element array.



Element x = A[r] is the pivot element

(a) The initial array

(b) The value 2 is "swapped with itself" and put in the partition of smaller values.

(c)-(d) The values 8 and 7 are added to the partition of larger values.

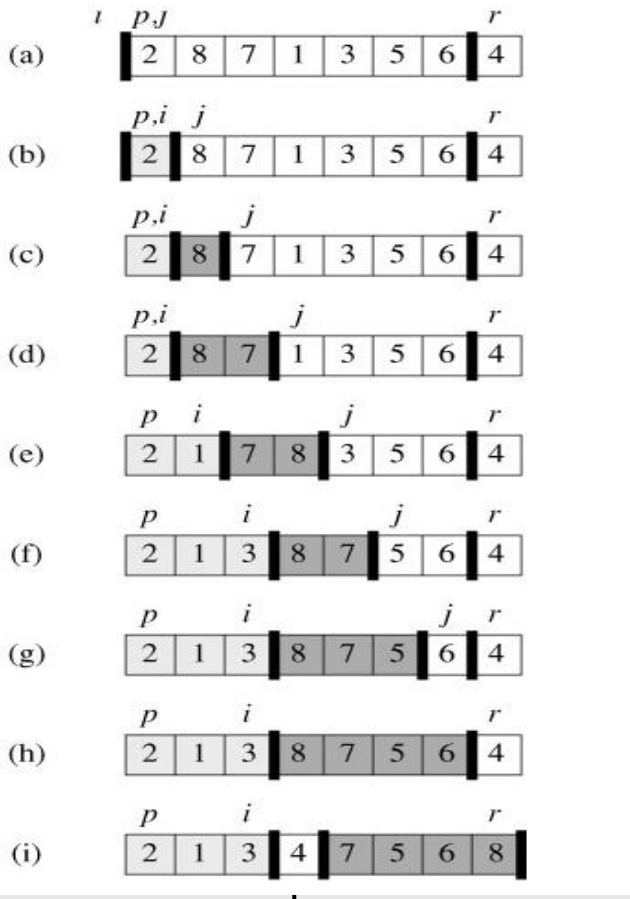
(e) The values 1 and 8 are swapped, and the smaller partition Grows.

(f) The values 3 and 7 are swapped, and the smaller partition grows.

(g)-(h) The larger partition grows to include 5 and 6 and the loop terminates.

(i) Pivot element is swapped so that it lies between the two partitions.

Operation of PARTITION on an 8-element array.



first partition with values <= pivot

second partition with values > pivot

white element - pivot.

Example

8	6	1	2	5	3	9	4

```
QUICKSORT (A,p,r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT (A, p, q-1)
    QUICKSORT (A, q+1, r)
```

PARTITION(*A*, *p*, *r*)

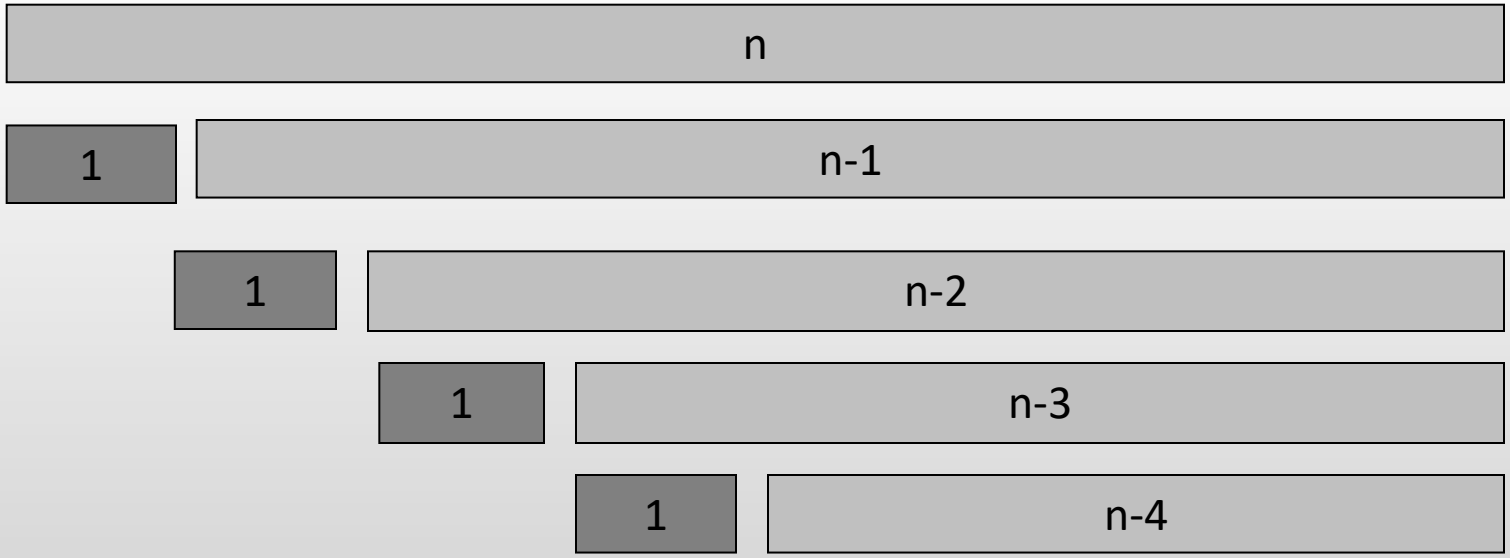
```
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] ≤ x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```

Analysis of Quick sort

The running time of quick sort depends on the partitioning of the sub arrays:

(a) Worst case partitioning (Unbalanced partitioning)

- Worst case occurs when the sub arrays are completely unbalanced. i.e. 0 elements in one sub array and $n - 1$ elements in the other sub array



Analysis of Quick sort

Worst case partitioning (Repeated Substituted method)

- Partitioning → $\Theta(n)$
- Recursive call on an array of size 0 → $T(0) = \Theta(1)$
- Recursive call on an array of size (n-1) → $T(n-1)$

Therefore **Recurrence** Equation is

$$T(n) = T(n-1) + T(0) + \Theta(n)$$

$$= T(n-1) + \Theta(n)$$

$$= T(n-2) + \Theta(n-1) + \Theta(n)$$

.....

$$= T(0) + \Theta(1) + \Theta(2) + \dots + \Theta(n-1) + \Theta(n)$$

$$= \sum_{k=1}^{n} \left(\Theta(k)\right) = \Theta \sum_{k=1}^{n} k = \Theta(n^2)$$

Worst case Running Time is $\Theta(n^2)$

Analysis of Quick sort

(b) Best case partitioning

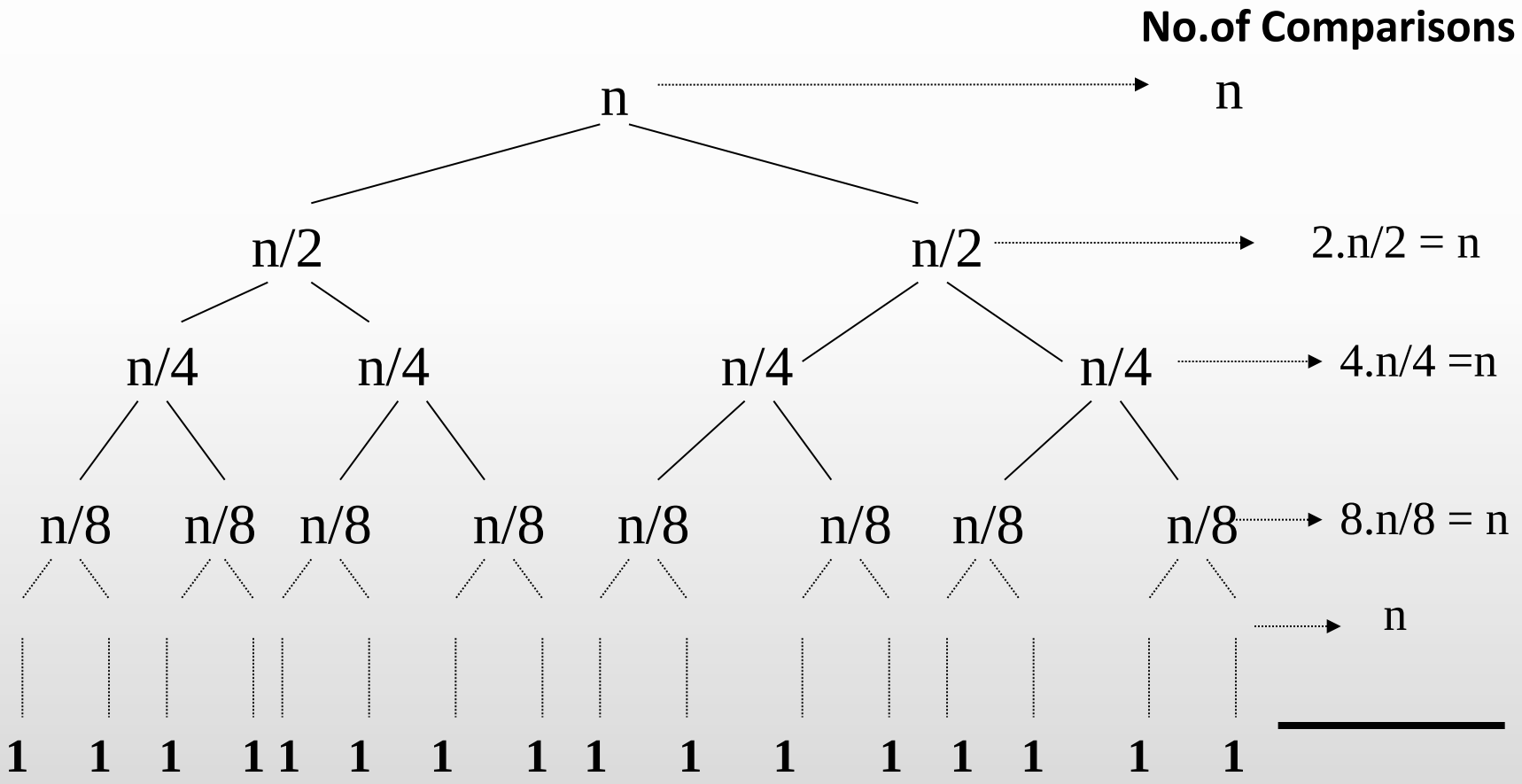
Best case occurs when PARTITION produces two sub arrays , one is of size (n-1)/2 and the other is of size (n-1)/2. In this case, quicksort runs much faster.

Recurrence equation is

$$T(n) = 2T(n/2) + \Theta(n)$$

Analysis of Quick Sort (with recursion tree)

Best Case:



Best case Running Time is Θ(n lg n)